

hw2

Yifeng Huang

This is the link to the repo: <https://github.com/YifengHuang-Bob/Homework2>

Problem 1: Random Walk with Momentum

a. Simulation function

```
#' Simulate a two-dimensional random walk with momentum
#'
```

```
#' @param n A positive integer giving the number of steps.
#'
```

```
#' @param p A numeric value between 0 and 1 giving the probability of
#'
```

```
#' preserving the previous direction.
#'
```

```
#' @return A numeric vector containing the final x and y coordinates.
momentum_walk <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) ||
      n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer.")
  }

  if (!is.numeric(p) || length(p) != 1 || is.na(p) ||
      p < 0 || p > 1) {
    stop("p must be a numeric value between 0 and 1.")
  }

  directions <- rbind(
    right = c(1, 0),
    left = c(-1, 0),
    up = c(0, 1),
    down = c(0, -1)
  )

  position <- c(0, 0)

  direction <- sample(1:4, 1)
  position <- position + directions[direction, ]

  if (n > 1) {
    for (i in 2:n) {
```

```

    if (runif(1) >= p) {
      direction <- sample(1:4, 1)
    }

    position <- position + directions[direction, ]
  }
}

return(position)
}

```

```
momentum_walk(10, 0.6)
```

```
[1] -5  3
```

```
momentum_walk(10, 0.6)
```

```
[1] 0 2
```

b. Implement the simulation

Version 1: Loop implementation

```

#' Simulate a two-dimensional random walk with momentum using a loop
#'
#' @param n A positive integer giving the number of steps.
#' @param p A numeric value between 0 and 1 giving the probability of
#'   preserving the previous direction.
#' @return A numeric vector containing the final x and y coordinates.
momentum_walk1 <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) ||
      n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer.")
  }

  if (!is.numeric(p) || length(p) != 1 || is.na(p) ||
      p < 0 || p > 1) {
    stop("p must be a numeric value between 0 and 1.")
  }
}

```

```

directions <- rbind(
  right = c(1, 0),
  left = c(-1, 0),
  up = c(0, 1),
  down = c(0, -1)
)

position <- c(0, 0)

direction <- sample(1:4, 1)
position <- position + directions[direction, ]

if (n > 1) {
  for (i in 2:n) {
    if (runif(1) >= p) {
      direction <- sample(1:4, 1)
    }

    position <- position + directions[direction, ]
  }
}

return(position)
}

```

Version 2: vapply implementation

```

#' Simulate a two-dimensional random walk with momentum using vapply
#'
#' @param n A positive integer giving the number of steps.
#' @param p A numeric value between 0 and 1 giving the probability of
#' preserving the previous direction.
#' @return A numeric vector containing the final x and y coordinates.
momentum_walk2 <- function(n, p) {
  if (!is.numeric(n) || length(n) != 1 || is.na(n) ||
      n < 1 || n %% 1 != 0) {
    stop("n must be a positive integer.")
  }

  if (!is.numeric(p) || length(p) != 1 || is.na(p) ||
      p < 0 || p > 1) {
    stop("p must be a numeric value between 0 and 1.")
  }
}

```

```

directions <- rbind(
  right = c(1, 0),
  left = c(-1, 0),
  up = c(0, 1),
  down = c(0, -1)
)

keep_direction <- runif(n - 1) < p

block_id <- c(1L, 1L + cumsum(!keep_direction))
n_blocks <- max(block_id)

block_direction <- vapply(
  seq_len(n_blocks),
  function(i) sample(1:4, 1),
  integer(1)
)

step_direction <- block_direction[block_id]

steps <- directions[step_direction, , drop = FALSE]

return(colSums(steps))
}

```

```
momentum_walk1(10, 0.6)
```

```
[1] 0 -10
```

```
momentum_walk2(10, 0.6)
```

```
[1] 2 -6
```

```
momentum_walk1(1000, 0.6)
```

```
[1] -68 -10
```

```
momentum_walk2(1000, 0.6)
```

```
[1] -38 -90
```

Version 1 simulates the random walk sequentially using a `for` loop. The first direction is selected uniformly at random. For each subsequent step, the previous direction is preserved with probability (p); otherwise, a new direction is selected uniformly from the four possible directions. The position is updated after every step.

Version 2 first generates the keep/change decisions for steps 2 through (n). These decisions divide the walk into blocks in which the direction remains unchanged. `vapply()` is used to randomly select one direction for each block. The block directions are then expanded to the individual steps, and `colSums()` is used to calculate the final position.

c. Reproducibility

```
set.seed(506)
momentum_walk1(10, 0.6)
```

```
[1] -2  0
```

```
set.seed(506)
momentum_walk1(10, 0.6)
```

```
[1] -2  0
```

```
set.seed(506)
momentum_walk1(1000, 0.6)
```

```
[1] -14 -30
```

```
set.seed(506)
momentum_walk1(1000, 0.6)
```

```
[1] -14 -30
```

I used `set.seed(506)` before each call to `momentum_walk1()`. Setting the same seed resets R's random-number generator to the same starting state. Therefore, the same sequence of random numbers is generated each time, so the simulation produces the same result when the function and its inputs are unchanged.

For $n = 10$ and $p = 0.6$, both runs produced the final position $(-2, 0)$. For $n = 1000$ and $p = 0.6$, both runs produced the final position $(-14, -30)$, demonstrating that the simulation is reproducible.

d. Does momentum matter?

i. Final positions and distances

```
set.seed(506)

positions <- rbind(
  p_0 = momentum_walk1(1000, 0),
  p_05 = momentum_walk1(1000, 0.5),
  p_09 = momentum_walk1(1000, 0.9)
)

distances <- sqrt(rowSums(positions^2))

results <- data.frame(
  p = c(0, 0.5, 0.9),
  x = positions[, 1],
  y = positions[, 2],
  distance = distances
)

results
```

	p	x	y	distance
p_0	0.0	3	-23	23.19483
p_05	0.5	-62	6	62.28965
p_09	0.9	-25	-155	157.00318

ii. Effect of increasing momentum

The final distances for $(p = 0)$, $(p = 0.5)$, and $(p = 0.9)$ were approximately 23.19, 62.29, and 157.00, respectively. In this simulation, increasing the momentum probability appears to increase the final distance from the starting point.

iii. Reasoning

A higher value of (p) makes the particle more likely to continue moving in the same direction as the previous step. This creates longer runs in the same direction and reduces cancellation between movements in opposite directions, so the particle tends to travel farther from the origin.

e. Monte Carlo simulation

i. Simulate 1,000 walks for each momentum value

```
set.seed(506)

p_values <- c(0, 0.25, 0.5, 0.75, 0.9)

distance_samples <- lapply(p_values, function(p) {
  replicate(1000, {
    position <- momentum_walk2(1000, p)
    sqrt(sum(position^2))
  })
})
```

ii. Calculate the mean final distance

```
mean_distances <- vapply(
  distance_samples,
  mean,
  numeric(1)
)
```

iii. Monte Carlo results

```
monte_carlo_results <- data.frame(
  p = p_values,
  mean_distance = mean_distances
)

monte_carlo_results
```

	p	mean_distance
1	0.00	27.60970
2	0.25	35.82068
3	0.50	47.92998
4	0.75	75.42132
5	0.90	121.74651

iv. Effect of momentum on mean final distance

The mean final distance increases as the momentum probability (p) increases. The estimated mean distance rises from approximately 27.61 when ($p = 0$) to approximately 121.75 when ($p = 0.9$).

This suggests that stronger momentum causes the particle to travel farther from the origin on average. As (p) increases, the particle is more likely to continue moving in the same direction for

consecutive steps, creating longer directional runs and reducing cancellation between opposite movements.

f. Compare computational performance

Benchmark for (n = 1000)

```
library(microbenchmark)

benchmark_1000 <- microbenchmark(
  loop_version = momentum_walk1(1000, 0.6),
  vapply_version = momentum_walk2(1000, 0.6),
  times = 100L,
  unit = "ms"
)

benchmark_1000
```

```
Unit: milliseconds
      expr      min       lq     mean  median      uq      max  neval
loop_version 1.3976 1.53625 2.187533 1.7274 2.50225 6.4437   100
vapply_version 0.9050 1.04610 1.426597 1.1356 1.74880 4.9916   100
```

Benchmark for (n = 100000)

```
benchmark_100000 <- microbenchmark(
  loop_version = momentum_walk1(100000, 0.6),
  vapply_version = momentum_walk2(100000, 0.6),
  times = 100L,
  unit = "ms"
)

benchmark_100000
```

```
Unit: milliseconds
      expr      min       lq     mean  median      uq      max  neval
loop_version 195.5066 228.4444 242.0791 237.1672 247.4196 326.0551   100
vapply_version 135.4071 152.6470 161.9117 158.2603 165.3771 235.1914   100
```


i. Which implementation is faster?

The `vapply` implementation is faster for both input sizes. For ($n = 1000$), the median execution time was approximately 1.73 ms for the loop version and 1.14 ms for the `vapply` version. For ($n = 100000$), the corresponding median execution times were approximately 237.17 ms and 158.26 ms.

ii. Does the difference become larger for the larger input?

Yes. For ($n = 1000$), the difference in median execution time was approximately 0.59 ms, whereas for ($n = 100000$), the difference was approximately 78.91 ms. Thus, the performance advantage of the `vapply` implementation becomes much more noticeable for the larger input.

iii. Why might the implementations have different execution times?

The loop implementation processes the walk one step at a time in an R `for` loop, repeatedly generating random numbers, checking the momentum condition, and updating the position. In contrast, the `vapply` implementation generates the keep/change decisions at once and performs much of the remaining work using vectorized operations such as `cumsum()`, indexing, and `colSums()`. These operations reduce the amount of step-by-step R-level loop overhead, which can lead to faster execution, especially when the number of steps is large.

Problem 2: Monte Carlo Simulation of Campus Bus Arrivals

Monte Carlo simulation

```
set.seed(506)

n_days <- 10000

passengers_6_to_8 <- rowSums(
  matrix(rpois(n_days * 3, lambda = 4), nrow = n_days)
)

passengers_9 <- rnorm(
  n_days,
  mean = 45,
  sd = 4
)

passengers_10_to_3 <- rowSums(
  matrix(rpois(n_days * 6, lambda = 15), nrow = n_days)
)
```

```

passengers_4 <- rnorm(
  n_days,
  mean = 55,
  sd = 5
)

passengers_5_to_10 <- rowSums(
  matrix(rpois(n_days * 6, lambda = 9), nrow = n_days)
)

daily_totals <- passengers_6_to_8 +
  passengers_9 +
  passengers_10_to_3 +
  passengers_4 +
  passengers_5_to_10

mean_daily_passengers <- mean(daily_totals)

mean_daily_passengers

```

```
[1] 256.1777
```

Estimated average daily passengers

The Monte Carlo simulation of 10,000 days gives an estimated average of approximately 256.18 passengers per day.

This result is very close to the theoretical expected total of

$$3(4) + 45 + 6(15) + 55 + 6(9) = 256,$$

which supports the accuracy of the simulation.

Problem 3: Linear Regression

a. Data preparation and de-identification

```

youtube <- read.csv(
  "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/
  2021/2021-03-02/youtube.csv"
)

```

```
dim(youtube)
```

```
[1] 247  25
```

```
names(youtube)
```

```
[1] "year"                "brand"  
[3] "superbowl_ads_dot_com_url" "youtube_url"  
[5] "funny"               "show_product_quickly"  
[7] "patriotic"           "celebrity"  
[9] "danger"              "animals"  
[11] "use_sex"             "id"  
[13] "kind"                "etag"  
[15] "view_count"          "like_count"  
[17] "dislike_count"       "favorite_count"  
[19] "comment_count"      "published_at"  
[21] "title"               "description"  
[23] "thumbnail"           "channel_title"  
[25] "category_id"
```

De-identified dataset

```
youtube_deidentified <- youtube[, !names(youtube) %in% c(  
  "brand",  
  "superbowl_ads_dot_com_url",  
  "youtube_url",  
  "id",  
  "etag",  
  "published_at",  
  "title",  
  "description",  
  "thumbnail",  
  "channel_title"  
)]  
  
dim(youtube_deidentified)
```

```
[1] 247  15
```

Result

The de-identified dataset contains 247 observations and 15 variables. I removed variables that could directly identify a specific commercial, including the brand, URLs, video ID, metadata, publication information, title, description, thumbnail, and channel title. The remaining variables do not directly identify an individual commercial and can be used for the subsequent analysis.

b. Exploring potential outcomes

Distribution summaries

```
engagement_vars <- c(
  "view_count",
  "like_count",
  "dislike_count",
  "favorite_count",
  "comment_count"
)

summary(youtube_deidentified[engagement_vars])
```

view_count	like_count	dislike_count	favorite_count
Min. : 10	Min. : 0	Min. : 0.0	Min. : 0
1st Qu.: 6431	1st Qu.: 19	1st Qu.: 1.0	1st Qu.: 0
Median : 41379	Median : 130	Median : 7.0	Median : 0
Mean : 1407556	Mean : 4146	Mean : 833.5	Mean : 0
3rd Qu.: 170016	3rd Qu.: 527	3rd Qu.: 24.0	3rd Qu.: 0
Max. : 176373378	Max. : 275362	Max. : 92990.0	Max. : 0
NAs : 16	NAs : 22	NAs : 22	NAs : 16

comment_count
Min. : 0.00
1st Qu.: 1.00
Median : 10.00
Mean : 188.64
3rd Qu.: 50.75
Max. : 9190.00
NAs : 25

```
sapply(
  youtube_deidentified[engagement_vars],
  function(x) c(
    missing = sum(is.na(x)),
    zero = sum(x == 0, na.rm = TRUE),
```

```

    unique = length(unique(x[!is.na(x)]))
  )
)

```

	view_count	like_count	dislike_count	favorite_count	comment_count
missing	16	22	22	16	25
zero	0	9	47	231	41
unique	227	170	72	1	91

Histograms of engagement variables

```

par(mfrow = c(2, 3))

hist(
  youtube_deidentified$view_count,
  main = "View count",
  xlab = "view_count"
)

hist(
  youtube_deidentified$like_count,
  main = "Like count",
  xlab = "like_count"
)

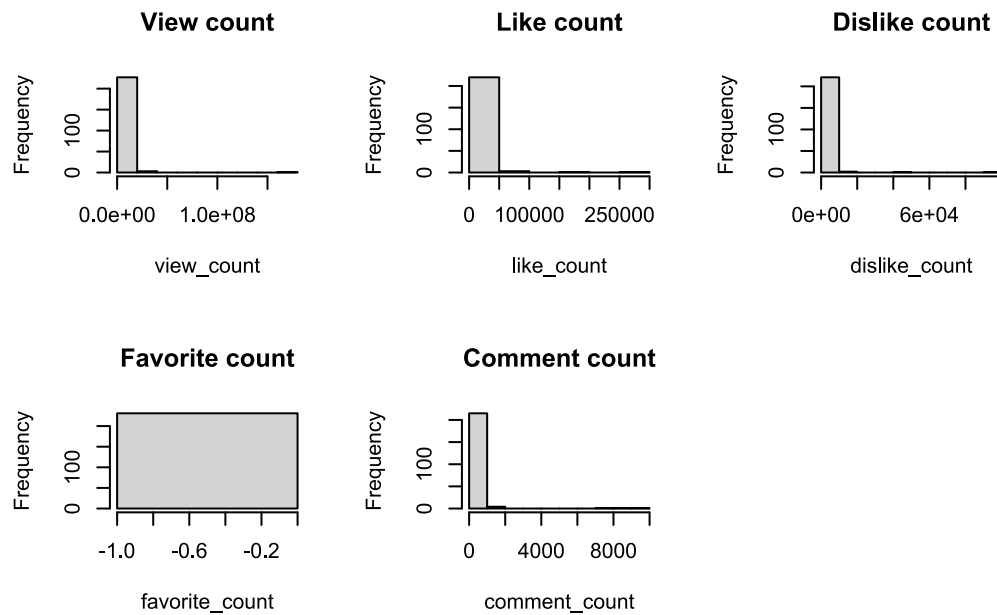
hist(
  youtube_deidentified$dislike_count,
  main = "Dislike count",
  xlab = "dislike_count"
)

hist(
  youtube_deidentified$favorite_count,
  main = "Favorite count",
  xlab = "favorite_count"
)

hist(
  youtube_deidentified$comment_count,
  main = "Comment count",
  xlab = "comment_count"
)

```

```
par(mfrow = c(1, 1))
```



Comparison of raw and log-transformed distributions

```
par(mfrow = c(2, 4))

hist(
  youtube_deidentified$view_count,
  main = "View count",
  xlab = "view_count"
)

hist(
  youtube_deidentified$like_count,
  main = "Like count",
  xlab = "like_count"
)

hist(
  youtube_deidentified$dislike_count,
  main = "Dislike count",
  xlab = "dislike_count"
)
```

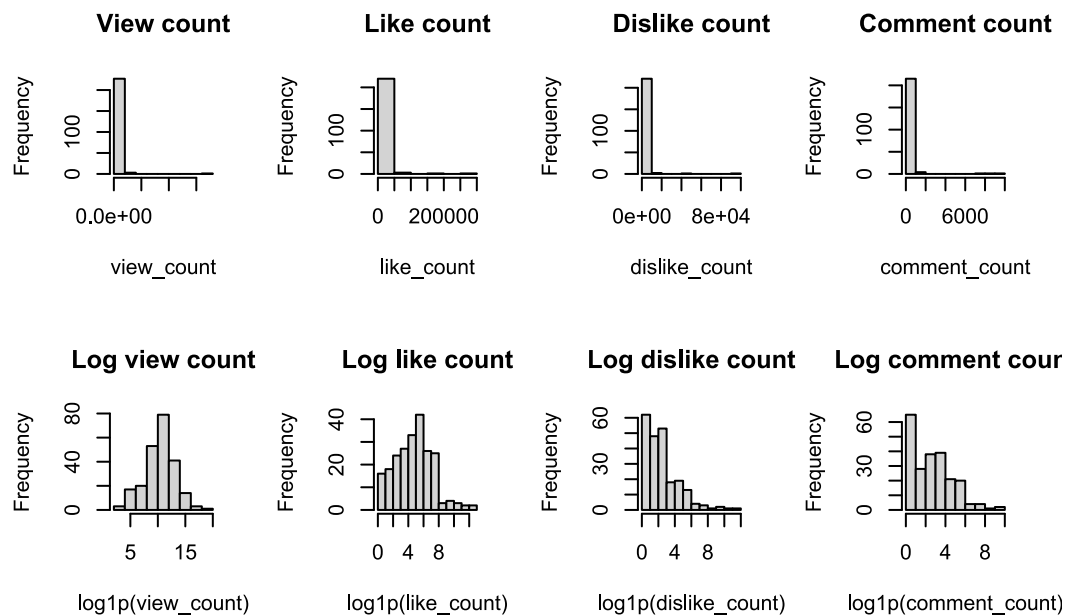
```
hist(
  youtube_deidentified$comment_count,
  main = "Comment count",
  xlab = "comment_count"
)

hist(
  loglp(youtube_deidentified$view_count),
  main = "Log view count",
  xlab = "loglp(view_count)"
)

hist(
  loglp(youtube_deidentified$like_count),
  main = "Log like count",
  xlab = "loglp(like_count)"
)

hist(
  loglp(youtube_deidentified$dislike_count),
  main = "Log dislike count",
  xlab = "loglp(dislike_count)"
)

hist(
  loglp(youtube_deidentified$comment_count),
  main = "Log comment count",
  xlab = "loglp(comment_count)"
)
```



```
par(mfrow = c(1, 1))
```

Transformed engagement variables

```
youtube_deidentified$log_view_count <- log1p(
  youtube_deidentified$view_count
)

youtube_deidentified$log_like_count <- log1p(
  youtube_deidentified$like_count
)

youtube_deidentified$log_dislike_count <- log1p(
  youtube_deidentified$dislike_count
)

youtube_deidentified$log_comment_count <- log1p(
  youtube_deidentified$comment_count
)
```

Outcome assessment

- **View count — Category 2:** The raw distribution is extremely right-skewed with several very large observations. The `log1p()` transformation substantially reduces the skewness, so `log_view_count` will be used as the outcome in subsequent analyses.

- **Like count — Category 2:** The raw distribution is strongly right-skewed and includes zero values. Applying `log1p()` reduces the skewness while allowing observations with zero likes to be retained. Therefore, `log_like_count` will be used in subsequent analyses.
- **Dislike count — Category 2:** The raw distribution is highly right-skewed and contains many zero values. The `log1p()` transformation improves the distribution and accommodates the zeros, so `log_dislike_count` will be used in subsequent analyses.
- **Favorite count — Category 3:** All non-missing observations have a value of zero, so this variable has no variation. It is therefore not appropriate to use as an outcome in a linear regression model.
- **Comment count — Category 2:** The raw distribution is strongly right-skewed and contains zero values. The `log1p()` transformation reduces the skewness and allows zero counts to be retained, so `log_comment_count` will be used in subsequent analyses.

c. Fitting linear regression models

Fit separate regression models

```
outcomes <- c(
  "log_view_count",
  "log_like_count",
  "log_dislike_count",
  "log_comment_count"
)

characteristics <- c(
  "funny",
  "show_product_quickly",
  "patriotic",
  "celebrity",
  "danger",
  "animals",
  "use_sex"
)
```

```
#' Fit one linear regression model
#'  

#' @param outcome Name of the engagement outcome variable.  

#' @param characteristic Name of the binary commercial characteristic.  

#' @param data Data frame containing the variables.  

#' @return A data frame containing the characteristic coefficient and p-value.  

fit_engagement_model <- function(outcome, characteristic, data) {
```

```

model_formula <- reformulate(
  c(characteristic, "year"),
  response = outcome
)

model <- lm(model_formula, data = data)

coefficient_table <- summary(model)$coefficients

data.frame(
  outcome = outcome,
  characteristic = characteristic,
  estimate = coefficient_table[2, "Estimate"],
  p_value = coefficient_table[2, "Pr(>|t|)"]
)
}

```

```

model_results <- do.call(
  rbind,
  lapply(outcomes, function(outcome) {
    do.call(
      rbind,
      lapply(characteristics, function(characteristic) {
        fit_engagement_model(
          outcome,
          characteristic,
          youtube_deidentified
        )
      })
    )
  })
)

rownames(model_results) <- NULL

model_results

```

	outcome	characteristic	estimate	p_value
1	log_view_count	funny	0.52984356	0.20510610
2	log_view_count	show_product_quickly	0.20898031	0.60072225
3	log_view_count	patriotic	0.13692497	0.78391866
4	log_view_count	celebrity	-0.01898556	0.96288885

5	log_view_count	danger	0.66678076	0.09406479
6	log_view_count	animals	-0.17326980	0.64669142
7	log_view_count	use_sex	-0.29629543	0.49051595
8	log_like_count	funny	0.36019066	0.34005646
9	log_like_count	show_product_quickly	0.16825538	0.64047730
10	log_like_count	patriotic	0.49311233	0.28968290
11	log_like_count	celebrity	0.26297830	0.47787826
12	log_like_count	danger	0.62526219	0.08031647
13	log_like_count	animals	0.02459257	0.94282914
14	log_like_count	use_sex	-0.32497453	0.40008928
15	log_dislike_count	funny	0.04148774	0.89580597
16	log_dislike_count	show_product_quickly	0.24290875	0.42097425
17	log_dislike_count	patriotic	0.66965405	0.08563747
18	log_dislike_count	celebrity	-0.28325315	0.36162721
19	log_dislike_count	danger	0.18514931	0.53794989
20	log_dislike_count	animals	-0.07378053	0.79742453
21	log_dislike_count	use_sex	-0.38888066	0.22927926
22	log_comment_count	funny	0.04746590	0.87934573
23	log_comment_count	show_product_quickly	0.34602814	0.24685774
24	log_comment_count	patriotic	0.51478460	0.16854269
25	log_comment_count	celebrity	0.22806291	0.45558834
26	log_comment_count	danger	0.10642262	0.72095533
27	log_comment_count	animals	-0.19176266	0.49906680
28	log_comment_count	use_sex	-0.34939714	0.27353634

Summary of findings

1. At the $\alpha = 0.05$ significance level, none of the seven commercial characteristics has a statistically significant association with any of the four engagement outcomes. All p-values are greater than 0.05.
2. Because there are no statistically significant associations, there are no significant positive or negative associations to report.
3. Among the seven characteristics, **danger** has the largest positive estimated association with transformed view counts. Its estimated coefficient is approximately 0.667, although this association is not statistically significant at the 0.05 level ($p = 0.094$).

d. Estimating regression coefficients using matrix algebra

Select the characteristic and prepare the data

```
analysis_data <- youtube_deidentified[
  complete.cases(
```

```

    youtube_deidentified[, c("log_view_count", "danger", "year")]
  ),
  c("log_view_count", "danger", "year")
]

analysis_data$danger <- as.numeric(analysis_data$danger)

```

Estimate coefficients using matrix algebra

```

y <- analysis_data$log_view_count

X <- cbind(
  intercept = 1,
  danger = analysis_data$danger,
  year = analysis_data$year
)

beta_hat <- solve(t(X) %*% X) %*%
  t(X) %*% y

beta_hat

```

```

              [,1]
intercept -31.39149243
danger      0.66678076
year       0.02065641

```

Compare with `lm()`

```

lm_model <- lm(
  log_view_count ~ danger + year,
  data = analysis_data
)

coef(lm_model)

```

```

(Intercept)      danger      year
-31.39149243   0.66678076   0.02065641

```

```

comparison <- data.frame(
  coefficient = c("(Intercept)", "danger", "year"),
  matrix_estimate = as.numeric(beta_hat),

```

```
lm_estimate = as.numeric(coef(lm_model))
)

comparison
```

	coefficient	matrix_estimate	lm_estimate
1 (Intercept)		-31.39149243	-31.39149243
2 danger		0.66678076	0.66678076
3 year		0.02065641	0.02065641

Conclusion

For the transformed view-count outcome, `danger` had the smallest p-value among the seven commercial characteristics in part 3-c, so it was selected as the binary predictor.

Using the matrix formula

$$\hat{\beta} = (X^T X)^{-1} X^T y,$$

the estimated coefficients are approximately -31.3915 for the intercept, 0.6668 for `danger`, and 0.0207 for `year`.

Fitting the same model with `lm()` gives the same coefficient estimates, confirming that the matrix-algebra calculation agrees with R's linear regression implementation up to numerical precision.